

OLTP vs OLAP

Table of Contents

- [Learning Objectives](#)
- [Theory: OLTP vs OLAP](#)
- [OLTP Deep Dive](#)
- [OLAP Deep Dive](#)
- [HTAP: The Hybrid Approach](#)
- [Data Warehousing Concepts](#)
- [ASCII Visual Diagrams](#)
- [Practical Examples](#)
- [PostgreSQL Commands](#)
- [SQL Examples](#)
- [Common Mistakes](#)
- [Best Practices](#)
- [Performance Considerations](#)
- [Interview Questions](#)
- [Interview Answers](#)
- [Hands-on Exercises](#)
- [Solutions](#)
- [Advanced Notes](#)
- [Cross-References](#)

Learning Objectives

By the end of this chapter, you will be able to:

- Clearly distinguish between OLTP and OLAP workloads with concrete examples
- Explain why the same database design does not work well for both
- Describe star schema and snowflake schema for data warehousing
- Understand how organizations build ETL pipelines to move data from OLTP to OLAP
- Know when to use PostgreSQL for OLAP and when to consider dedicated column stores

Theory: OLTP vs OLAP

Dimension	OLTP	OLAP
Full Name	Online Transaction Processing	Online Analytical Processing
Purpose	Day-to-day operations	Business intelligence, analytics
Users	Thousands of clerks/customers	Hundreds of analysts
Query Type	Simple, short, CRUD	Complex, long-running, aggregations
Data Volume	Current data, GBs	Historical data, TBs-PBs
Response Time	Milliseconds	Seconds to minutes

Data Model	Normalized (3NF)	Denormalized (star/snowflake schema)
Operations	INSERT/UPDATE/DELETE heavy	SELECT/aggregate heavy
Transaction Size	Small (few rows)	Large (millions of rows)
Concurrency	High (1000s of users)	Low (dozens of analysts)
Database Design	Row-oriented	Column-oriented
Examples	PostgreSQL, MySQL, Oracle	Redshift, BigQuery, Snowflake, ClickHouse

OLTP Deep Dive

OLTP systems are the transactional heartbeat of a business. Every customer purchase, bank transfer, inventory update, and login event is an OLTP transaction.

Characteristics

1. **High write throughput:** Hundreds to thousands of INSERT/UPDATE/DELETE per second
2. **Short transactions:** Each transaction touches a few rows, completes quickly
3. **ACID compliance:** Critical — a bank transfer must either fully complete or fully roll back
4. **Normalized schema:** Reduces write amplification (updating one value updates one place)
5. **Row-oriented storage:** Entire rows are read/written together
6. **Point queries:** Fetch customer #1234, not "all customers in region X"

OLTP Schema Characteristics

- Many small, normalized tables (3NF)
- Many foreign keys and constraints
- Indexes on PKs and frequently queried columns
- Short, frequent transactions

PostgreSQL as OLTP

PostgreSQL is a world-class OLTP database:

- MVCC (Multi-Version Concurrency Control) for high concurrency
- WAL (Write-Ahead Log) for durability
- Row-level locking
- Comprehensive constraint system
- Connection pooling with PgBouncer

OLAP Deep Dive

OLAP systems answer business questions by aggregating large amounts of historical data.

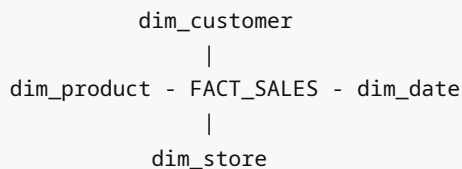
Characteristics

1. **Read-heavy:** Analysts run complex SELECT queries; rarely write data
2. **Long-running queries:** Scanning millions/billions of rows for aggregations
3. **Denormalized schema:** Fewer JOINS = faster queries at scale
4. **Column-oriented storage:** Only needed columns are read from disk
5. **Historical data:** Years of transaction history

6. **Analytical queries:** Trends, forecasts, drill-down analysis

Star Schema

The most common OLAP schema. A central **fact table** surrounded by **dimension tables**.

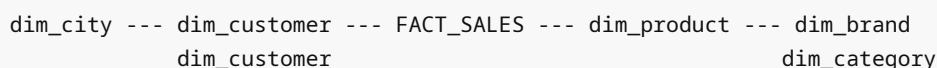


Fact table: Stores measurable events (sales, clicks, logins). Contains FK references to dimensions and numeric measures (amount, quantity, duration).

Dimension tables: Descriptive context (who, what, when, where, how). Wide tables with many descriptive attributes.

Snowflake Schema

Extension of star schema where dimension tables are normalized into sub-dimensions:



Trade-off: Less redundancy vs. more JOINS.

HTAP: The Hybrid Approach

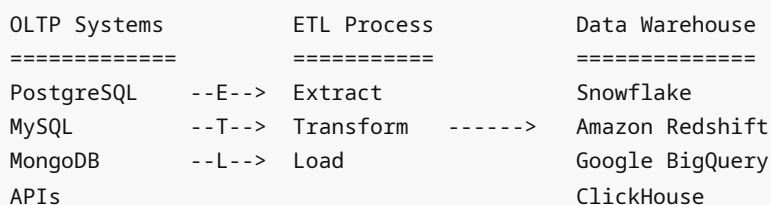
HTAP (Hybrid Transactional/Analytical Processing) systems handle both workloads simultaneously.

Options:

- **TiDB / SingleStore:** Purpose-built HTAP databases
- **PostgreSQL + TimescaleDB:** Columnar compression for analytics
- **PostgreSQL + citus:** Distributed PostgreSQL
- **Read replicas:** Route OLAP queries to a replica, OLTP to primary

Data Warehousing Concepts

ETL Pipeline



- **Extract:** Pull data from source systems
- **Transform:** Clean, deduplicate, aggregate, reshape
- **Load:** Insert into the warehouse

ELT (Modern Approach)

Load raw data first, then transform using the warehouse's compute power. Common with cloud data warehouses (BigQuery, Snowflake).

Data Lake

Raw, unprocessed data stored cheaply (S3, GCS). Queried with tools like Presto, Spark, Athena. A **Lakehouse** (Delta Lake, Iceberg) adds ACID transactions to a data lake.

ASCII Visual Diagrams

Diagram 1: OLTP vs OLAP Workload Comparison

OLTP WORKLOAD	OLAP WORKLOAD
=====	=====
Query: "Get order #1234"	Query: "Total sales by region, Q4 2024"
v	v
SELECT * FROM orders	SELECT region, SUM(amount)
WHERE order_id = 1234	FROM fact_sales fs
	JOIN dim_date d ON fs.date_id = d.date_id
	JOIN dim_store s ON fs.store_id = s.store_id
	WHERE d.quarter = 4 AND d.year = 2024
	GROUP BY region
	ORDER BY SUM(amount) DESC
Rows touched: 1	Rows touched: 50,000,000
Time: < 1ms	Time: 5-30 seconds
Frequency: 10,000/sec	Frequency: 10/hour

Diagram 2: Star Schema



```

--> sale_id (PK)
    date_id      FK --> DIM_DATE
    product_id   FK --> DIM_PRODUCT
    customer_id  FK --> DIM_CUSTOMER
    store_id     FK --> DIM_STORE
    units_sold   (measure)
    revenue      (measure)
    cost         (measure)
    profit       (computed)
        |
        DIM_STORE
        =====
        store_id (PK)
        store_name
        city, state
        region

```

Diagram 3: Row vs Column Storage

ROW-ORIENTED (PostgreSQL default)	COLUMN-ORIENTED (Parquet, Redshift)
=====	=====
Reading a row = reading one block	Reading a column = one disk scan
[id name age city salary]	[id col] [name col] [age col] ...
Row 1: [1 Alice 30 NYC 90000]	[1] [Alice] [30]
Row 2: [2 Bob 25 LA 80000]	[2] [Bob] [25]
Row 3: [3 Carol 35 NYC 95000]	[3] [Carol] [35]
For query: SELECT AVG(salary)	Only reads [salary col]
Must read ALL columns	Skips name, age, city columns
= 100% data read	= 20% data read (1 of 5 cols)
BEST FOR: OLTP (point queries)	BEST FOR: OLAP (aggregations)

Practical Examples

Example 1: E-Commerce OLTP to OLAP

OLTP (PostgreSQL):

- Processes 50,000 orders/day
- Tables: customers, orders, order_items, products, inventory
- Normalized to 3NF
- Primary key lookups, small transactions

OLAP (Redshift/BigQuery):

- Nightly ETL loads yesterday's orders
- Star schema: fact_orders + dim_customer + dim_product + dim_date
- Analysts run: "Compare Black Friday sales 2023 vs 2024"
- Query scans 10M rows in 8 seconds

Example 2: Banking Analytics

OLTP: Core banking system (Temenos, Oracle FLEXCUBE)

- Real-time account balances
- Transaction processing
- Strict ACID required

OLAP: Data warehouse (Snowflake)

- Risk reporting
- Customer segmentation
- Regulatory compliance reports
- ETL runs nightly at 2 AM

PostgreSQL Commands

```
-- Check table access statistics (OLTP vs OLAP indicator)
SELECT
    relname,
    seq_scan,      -- Full table scans (bad for OLTP, expected for OLAP)
    idx_scan,      -- Index scans (good for OLTP)
    n_tup_ins,     -- Inserts
    n_tup_upd,     -- Updates
    n_tup_del      -- Deletes
FROM pg_stat_user_tables
ORDER BY seq_scan DESC;

-- Check query execution mode
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT region, SUM(revenue)
FROM fact_sales
GROUP BY region;
```

SQL Examples

Example 1: OLTP-Style Query (Point Lookup)

```
-- OLTP: Get a specific order with all details (fast with indexes)
SELECT
    o.order_id,
    o.order_date,
    c.first_name || ' ' || c.last_name AS customer,
    o.total,
    o.status
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.order_id = 54321;
```

Example 2: OLAP-Style Query (Aggregation)

```
-- OLAP: Monthly revenue trend for the past year
SELECT
    DATE_TRUNC('month', order_date) AS month,
    COUNT(*) AS order_count,
    SUM(total) AS revenue,
    AVG(total) AS avg_order_value
FROM orders
WHERE order_date >= NOW() - INTERVAL '12 months'
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY month;
```

Example 3: Building a Star Schema in PostgreSQL

```
-- Dimension: Date
CREATE TABLE dim_date (
    date_id INT PRIMARY KEY,
    full_date DATE,
    day INT,
    month INT,
    month_name VARCHAR(20),
    quarter INT,
    year INT,
    is_weekend BOOLEAN
);

-- Dimension: Product
CREATE TABLE dim_product (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(200),
    brand VARCHAR(100),
    category VARCHAR(100),
    subcategory VARCHAR(100)
);

-- Dimension: Customer
CREATE TABLE dim_customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(200),
    segment VARCHAR(50),
    region VARCHAR(50),
    country VARCHAR(50)
);

-- Fact Table: Sales
CREATE TABLE fact_sales (
    sale_id BIGSERIAL PRIMARY KEY,
    date_id INT REFERENCES dim_date(date_id),
    product_id INT REFERENCES dim_product(product_id),
```

```

customer_id INT REFERENCES dim_customer(customer_id),
units_sold  INT,
unit_price  DECIMAL(10, 2),
revenue     DECIMAL(12, 2),
cost        DECIMAL(12, 2),
profit      DECIMAL(12, 2)
);

```

Example 4: OLAP Aggregation on Star Schema

```

-- Revenue by product category and quarter
SELECT
    dd.year,
    dd.quarter,
    dp.category,
    SUM(fs.revenue) AS total_revenue,
    SUM(fs.profit)  AS total_profit,
    COUNT(fs.sale_id) AS transactions
FROM fact_sales fs
JOIN dim_date    dd ON fs.date_id    = dd.date_id
JOIN dim_product dp ON fs.product_id = dp.product_id
WHERE dd.year = 2024
GROUP BY dd.year, dd.quarter, dp.category
ORDER BY dd.quarter, total_revenue DESC;

```

Example 5: ETL Pattern — Load Dimension Table

```

-- SCD Type 1 (Slowly Changing Dimension): Overwrite
INSERT INTO dim_customer (customer_id, customer_name, segment, region, country)
SELECT
    c.customer_id,
    c.first_name || ' ' || c.last_name,
    seg.segment_name,
    r.region_name,
    r.country
FROM customers c
JOIN customer_segments seg ON c.segment_id = seg.id
JOIN regions r            ON c.region_id  = r.id
ON CONFLICT (customer_id) DO UPDATE SET
    customer_name = EXCLUDED.customer_name,
    segment       = EXCLUDED.segment,
    region        = EXCLUDED.region;

```

Example 6: ETL Pattern — Load Fact Table

```

-- Load yesterday's sales into fact table
INSERT INTO fact_sales (date_id, product_id, customer_id, units_sold, unit_price,
revenue, cost, profit)

```

```

SELECT
    TO_CHAR(o.order_date, 'YYYYMMDD')::INT AS date_id,
    oi.product_id,
    o.customer_id,
    oi.quantity,
    oi.unit_price,
    oi.quantity * oi.unit_price AS revenue,
    oi.quantity * p.cost_price AS cost,
    (oi.quantity * oi.unit_price) - (oi.quantity * p.cost_price) AS profit
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p    ON oi.product_id = p.product_id
WHERE o.order_date = CURRENT_DATE - 1;

```

Example 7: Materialized View for OLAP in PostgreSQL

```

-- Pre-compute expensive aggregation
CREATE MATERIALIZED VIEW mv_monthly_sales AS
SELECT
    DATE_TRUNC('month', order_date) AS month,
    product_id,
    SUM(quantity * unit_price)      AS revenue,
    COUNT(DISTINCT order_id)       AS orders
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY DATE_TRUNC('month', order_date), product_id
WITH DATA;

-- Create index on materialized view
CREATE INDEX ON mv_monthly_sales (month, product_id);

-- Refresh daily
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_monthly_sales;

```

Example 8: Top-N per Group (OLAP Pattern)

```

-- Top 3 products by revenue in each category
SELECT *
FROM (
    SELECT
        dp.category,
        dp.product_name,
        SUM(fs.revenue) AS total_revenue,
        RANK() OVER (PARTITION BY dp.category ORDER BY SUM(fs.revenue) DESC) AS rank
    FROM fact_sales fs
    JOIN dim_product dp ON fs.product_id = dp.product_id
    GROUP BY dp.category, dp.product_name
) ranked
WHERE rank <= 3;

```

Example 9: Year-over-Year Comparison

```
-- YoY revenue comparison
SELECT
    dd.month,
    SUM(CASE WHEN dd.year = 2024 THEN fs.revenue ELSE 0 END) AS revenue_2024,
    SUM(CASE WHEN dd.year = 2023 THEN fs.revenue ELSE 0 END) AS revenue_2023,
    ROUND(
        (SUM(CASE WHEN dd.year = 2024 THEN fs.revenue ELSE 0 END) -
         SUM(CASE WHEN dd.year = 2023 THEN fs.revenue ELSE 0 END)) /
        NULLIF(SUM(CASE WHEN dd.year = 2023 THEN fs.revenue ELSE 0 END), 0) * 100,
        2) AS yoy_growth_pct
FROM fact_sales fs
JOIN dim_date dd ON fs.date_id = dd.date_id
WHERE dd.year IN (2023, 2024)
GROUP BY dd.month
ORDER BY dd.month;
```

Example 10: Running Total (Cumulative Sum)

```
SELECT
    month,
    revenue,
    SUM(revenue) OVER (ORDER BY month) AS cumulative_revenue
FROM mv_monthly_sales
WHERE product_id = 1
ORDER BY month;
```

Common Mistakes

Mistake 1: Running OLAP Queries on the OLTP Database

Running heavy analytical queries against your production OLTP database starves transactional queries of resources, causing latency spikes for customers.

Fix: Use read replicas for analytics, or build a separate data warehouse.

Mistake 2: Normalizing the Data Warehouse

OLAP schemas should be denormalized (star schema). Normalizing a data warehouse forces too many JOINS at query time across billions of rows.

Mistake 3: Not Partitioning Fact Tables

Fact tables grow without bound. Without partitioning, queries scan the entire table.

```
-- Partition fact_sales by year
CREATE TABLE fact_sales (
    sale_id    BIGSERIAL,
    sale_date  DATE NOT NULL,
```

```
...
) PARTITION BY RANGE (sale_date);

CREATE TABLE fact_sales_2024 PARTITION OF fact_sales
FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');
```

Mistake 4: Using OLTP Indexes for OLAP

B-tree indexes on individual columns are great for OLTP point queries. For OLAP, consider:

- **Partial indexes** on date ranges
- **BRIN indexes** for naturally ordered data (dates)
- **Column store extensions** (cstore_fdw, pg_mooncake)

Mistake 5: Forgetting to Populate the Date Dimension

The date dimension must be pre-populated for all dates in your analysis range. Joining on incomplete date dimensions produces wrong results.

Best Practices

1. **Separate OLTP from OLAP:** Use different infrastructure for different workloads.
2. **Use star schema for the data warehouse:** Denormalize for query speed.
3. **Partition large fact tables by date:** Essential for performance.
4. **Build incremental ETL:** Only load new/changed data, not full refreshes.
5. **Use materialized views in PostgreSQL for lightweight OLAP:** Great for smaller datasets.
6. **Monitor query patterns:** Use pg_stat_statements to identify OLAP queries running on OLTP.
7. **Pre-aggregate where possible:** Hourly/daily/monthly rollup tables speed up common queries.

Performance Considerations

- **Column stores** (Redshift, ClickHouse) outperform row stores by 10-100x for aggregation queries
- **Partial aggregation:** Compute partial aggregates at ETL time to avoid recomputation
- **Query parallelism:** PostgreSQL supports parallel query execution
(max_parallel_workers_per_gather)
- **BRIN indexes** for fact tables ordered by date: very small, very fast range scans
- **Table statistics:** Run `ANALYZE` after bulk loads to keep the query planner accurate

```
-- Enable parallel query for OLAP workloads
SET max_parallel_workers_per_gather = 4;
SET enable_parallel_hash = ON;

-- Analyze after bulk load
ANALYZE fact_sales;
```

Interview Questions

1. What is the difference between OLTP and OLAP?
2. Why can't you run OLAP queries efficiently on an OLTP schema?
3. What is a star schema?

4. What is the difference between a fact table and a dimension table?
 5. What is ETL and what are its three steps?
 6. What is a Slowly Changing Dimension (SCD)?
 7. How would you use PostgreSQL for OLAP workloads?
 8. What is HTAP?
-

Interview Answers

Q1: OLTP handles real-time transactional data with many short, ACID-compliant writes (millisecond response). OLAP handles historical analytical queries with complex aggregations over large datasets (seconds-to-minutes response). They differ in schema design, storage engine, hardware requirements, and user types.

Q6: Slowly Changing Dimensions (SCD)

Dimension tables change over time (a customer moves cities, a product changes category). SCDs handle this:

- **Type 1:** Overwrite the old value (no history)
- **Type 2:** Add a new row with effective/expiry dates (full history)
- **Type 3:** Add a "previous value" column (limited history)
- **Type 4:** Maintain a separate history table

Q7: PostgreSQL for OLAP

PostgreSQL can handle moderate OLAP workloads using: materialized views with indexes, table partitioning by date, parallel query execution, BRIN indexes on fact tables, columnar storage extensions (pg_mooncake, cstore_fdw), and the TimescaleDB extension for time-series analytics.

Hands-on Exercises

Exercise 1: Build a Star Schema

Create a star schema for an online store with: fact_orders, dim_date, dim_customer, dim_product. Populate each dimension and load 1000 fact rows.

Exercise 2: Write OLAP Queries

Using your star schema, write queries for: monthly revenue by product category, top 5 customers by lifetime value, year-over-year growth.

Exercise 3: Materialized View

Create a materialized view that pre-aggregates daily revenue by product. Add an index. Write a query that uses the view.

Exercise 4: Partitioning

Create a partitioned version of fact_orders, partitioned by year. Insert data and verify PostgreSQL uses partition pruning.

Exercise 5: Query Plan Analysis

Run an OLAP query with and without indexes. Use EXPLAIN ANALYZE to compare execution plans.

Solutions

Solution 1 (Populate Date Dimension)

```
-- Generate all dates for 2024
INSERT INTO dim_date (date_id, full_date, day, month, month_name, quarter, year,
is_weekend)
SELECT
    TO_CHAR(d, 'YYYYMMDD')::INT,
    d::DATE,
    EXTRACT(DAY FROM d)::INT,
    EXTRACT(MONTH FROM d)::INT,
    TO_CHAR(d, 'Month'),
    EXTRACT(QUARTER FROM d)::INT,
    EXTRACT(YEAR FROM d)::INT,
    EXTRACT(DOW FROM d) IN (0, 6)
FROM generate_series('2024-01-01'::DATE, '2024-12-31'::DATE, '1 day') d;
```

Advanced Notes

columnar Storage in PostgreSQL

PostgreSQL is row-oriented by default. For OLAP, consider:

- **pg_mooncake:** Columnar storage natively in PostgreSQL
- **TimescaleDB:** Columnar compression for time-series
- **Hydra:** Columnar tables in PostgreSQL
- **DuckDB + FDW:** Analytical engine querying PostgreSQL data

Lambda Architecture

A data architecture pattern combining:

- **Batch layer:** Full historical recomputation (Spark, Hive)
- **Speed layer:** Real-time stream processing (Kafka Streams, Flink)
- **Serving layer:** Merged results for queries

Kappa Architecture

Simplified: everything through streaming. The batch layer is replaced with reprocessing from a log (Kafka topic).

Cross-References

- **Previous:** [03 - Relational Databases](#)
 - **Next:** [05 - CAP Theorem](#)
 - **Related:** [02 - Types of Databases](#)
 - **See Also:** [04 Advanced SQL/05 window functions.md](#) — Window functions for OLAP queries
 - **See Also:** [07 Performance/04 partitioning.md](#) — Table partitioning
 - **See Also:** [09 Administration/03 monitoring.md](#) — pg_stat_statements
-

